Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Tarea - TIA-03 (Unidad 2 - Tarea 1)

Implementación de la estructura completa de una base de datos (DDL)

- **Tarea en Equipo**
- **Peso: 15%**
- **Práctica. Caso de Estudio**
- **Diccionario de Datos Físico - DBMS - PostgreSQL - pgAdmin4**

MIEMBROS DEL EQUIPO:

- Yuli Tatiana Blanco Nieto
- Juan David Restrepo Bedoya

Descripción de la Tarea

Hemos identificado una necesidad clara: mejorar la comunicación y la interacción entre los estudiantes. Aunque tenemos plataformas institucionales, a menudo son formales y no facilitan la conexión casual, el intercambio de conocimientos entre pares, la formación de grupos de estudio o la organización de eventos informales.

El objetivo con este reto es diseñar la base de datos que soportará una red social con estas características. Este es un problema real que resolverán usando las herramientas y conceptos aprendidos. El objetivo de este reto es tomar el modelo lógico que ya crearon para la red social estudiantil pascualina y lo implementen como un modelo físico funcional en un Sistema de gestión de base de datos (SGBD) real, aplicando rigurosamente las reglas y estándares del lenguaje de definición de datos (DDL).

Para lograrlo, aplicarán todo lo aprendido con el libro ***"Estructura, reglas y estándares",*** que hace parte de los recursos educativos de la unidad; así como todo el material compartido por el docente y la plataforma.

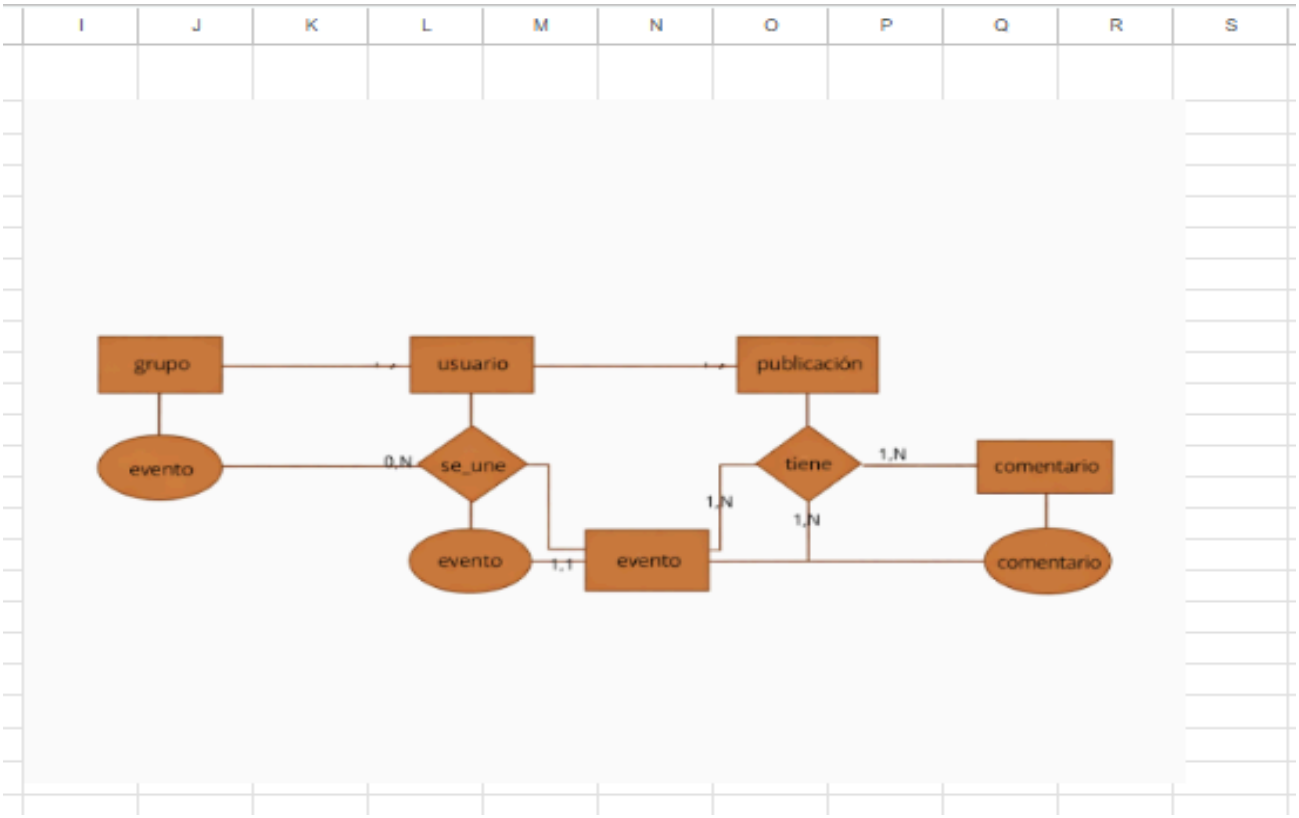
Es importante al identificación correcta de los tipos de datos del SGBD en el cual implementará la base de datos. Los estudiantes deberán elaborar un Diccionario de Datos Físico, coherente y consistente al Diccionario de Datos Genérico elaborado previamente. Este diccionario físico debe documentar cada tabla y cada campo, indicando nombre, tipo de dato, si es clave primaria, clave foránea, si es obligatorio (no nulo) o único, y su descripción funcional dentro del sistema. Por lo tanto, se recomienda mucha atención en la conversión de los tipos de datos genéricos en los tipos de datos del SGBD.

Informe con resultados

1.- Diagrama Entidad-Relación de Chen (corregido)

Para el desarrollo de esta actividad se realizó la corrección del diagrama Entidad-Relación (ER) de Chen, ajustando errores presentes en la versión inicial. Se identificaron las entidades principales del sistema como usuario, grupo, publicación, comentario y evento, junto con sus respectivas relaciones.Se

eliminaron los atributos del diagrama con el fin de cumplir con los requerimientos establecidos, dejando únicamente las entidades y relaciones. Además, se organizaron las relaciones entre entidades, como la interacción entre usuario y grupo, usuario y publicación, y publicación con comentario.Este proceso permitió obtener un modelo conceptual más claro, estructurado y coherente con el funcionamiento de la red social estudiantil.



2.- Inventario de Tablas del Diccionario de Datos Genérico (corregido)

A partir del modelo lógico corregido, se realizó el inventario de las tablas que conforman la base de datos. Este inventario incluye entidades como usuario, programa_academico, publicacion, grupo, evento, comentario, entre otras.Se utilizó una nomenclatura en formato snake_case para los nombres de las tablas, lo que facilita su comprensión y mantiene consistencia en el diseño.Este inventario sirve como base para la construcción del diccionario de datos físico y la implementación en PostgreSQL.

	A	B	C	D
1	**Tablas resultantes del Diccionario Genérico (Modelo Lógico)**			
2	#	Tabla	Descripción	Observaciones
3	1	usuario	Información de usuarios	Principal
4	2	programa_academico	Programas académicos	Relación estudiante
5	3	estudiante	Datos estudiantes	Subtipo
6	4	docente	Datos docentes	Subtipo
7	5	empleado	Datos empleados	Subtipo
8	6	egresado	Datos egresados	Subtipo
9	7	publico_general	Usuarios externos	Subtipo
10	8	habilidad	Habilidades	N:M
11	9	interes	Intereses	N:M
12	10	usuario_habilidad	Relación	Intermedia
13	11	usuario_interes	Relación	Intermedia
14	12	grupo	Grupos	Comunidad
15	13	publicacion	Publicaciones	Social
16	14	comentario	Comentarios	Recursiva
17	15	reaccion	Reacciones	Única
18	16	seguimiento	Seguidores	Reflexiva
19	17	miembro_grupo	Miembros	N:M
20	18	evento	Eventos	Comunidad
21	19	asistencia_evento	Asistencia	N:M
22	20	producto	Productos	Marketplace
23	21	intercambio	Transacciones	Relación
24	22	empresa	Empresas	Independiente
25	23	oferta_laboral	Ofertas	Empleo
26	24	postulacion	Postulaciones	N:M
27	25	mensaje	Mensajes	Chat
28	26	notificacion	Notificaciones	Sistema
29	27	reporte	Reportes	Moderación
30				

3.- Tabla Comparativa de tipos de Datos

Se realizó una comparación entre los tipos de datos definidos en el modelo lógico y los tipos de datos utilizados en PostgreSQL.Por ejemplo, los identificadores se implementaron como SERIAL, los campos de texto como VARCHAR o TEXT dependiendo de su longitud, y las fechas como TIMESTAMP.Esta comparación permitió adaptar correctamente el modelo lógico al modelo físico, garantizando compatibilidad con el sistema gestor de bases de datos.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

A	B
Tipos de Datos - Tabla Comparativa	
Tipo General	PostgreSQL
AUTOINCREMENTABLE	SERIAL
UUID	UUID
ENTERO	INTEGER
DECIMAL	DECIMAL(10,2)
LÓGICO	BOOLEAN
TEXTO	TEXT
CARACTER	VARCHAR
FECHA	DATE
FECHA_HORA	TIMESTAMP
IMAGEN	BYTEA / URL
AUDIO	BYTEA / URL
URL	VARCHAR(255)
ENUM	VARCHAR + CHECK
TEXTO_CORTO	VARCHAR(50)
BINARIO	BYTEA
JSON	JSONB

4.- Diccionario de Datos Físico (SGBD PostgreSQL)

El diccionario de datos físico fue construido en PostgreSQL a partir del modelo lógico previamente definido. En este se documentaron todas las tablas del sistema junto con sus atributos, tipos de datos y restricciones.Se implementaron tablas como usuario, estudiante, docente, publicacion, grupo, evento, producto, empresa, entre otras, las cuales representan las diferentes funcionalidades del sistema.Se utilizaron tipos de datos como VARCHAR para textos cortos, TEXT para descripciones extensas, JSONB para almacenamiento flexible de datos como contenido multimedia, y SERIAL para claves primarias.Además, se aplicaron reglas de nomenclatura en formato snake_case para tablas y campos, garantizando consistencia.

Adicionalmente, en la implementación del modelo físico se tomaron decisiones importantes en cuanto al uso de tipos de datos y restricciones. Se utilizó el tipo SERIAL para la generación automática de identificadores únicos en las claves primarias. Los campos de texto corto fueron definidos como VARCHAR, mientras que para contenidos más extensos se empleó TEXT.

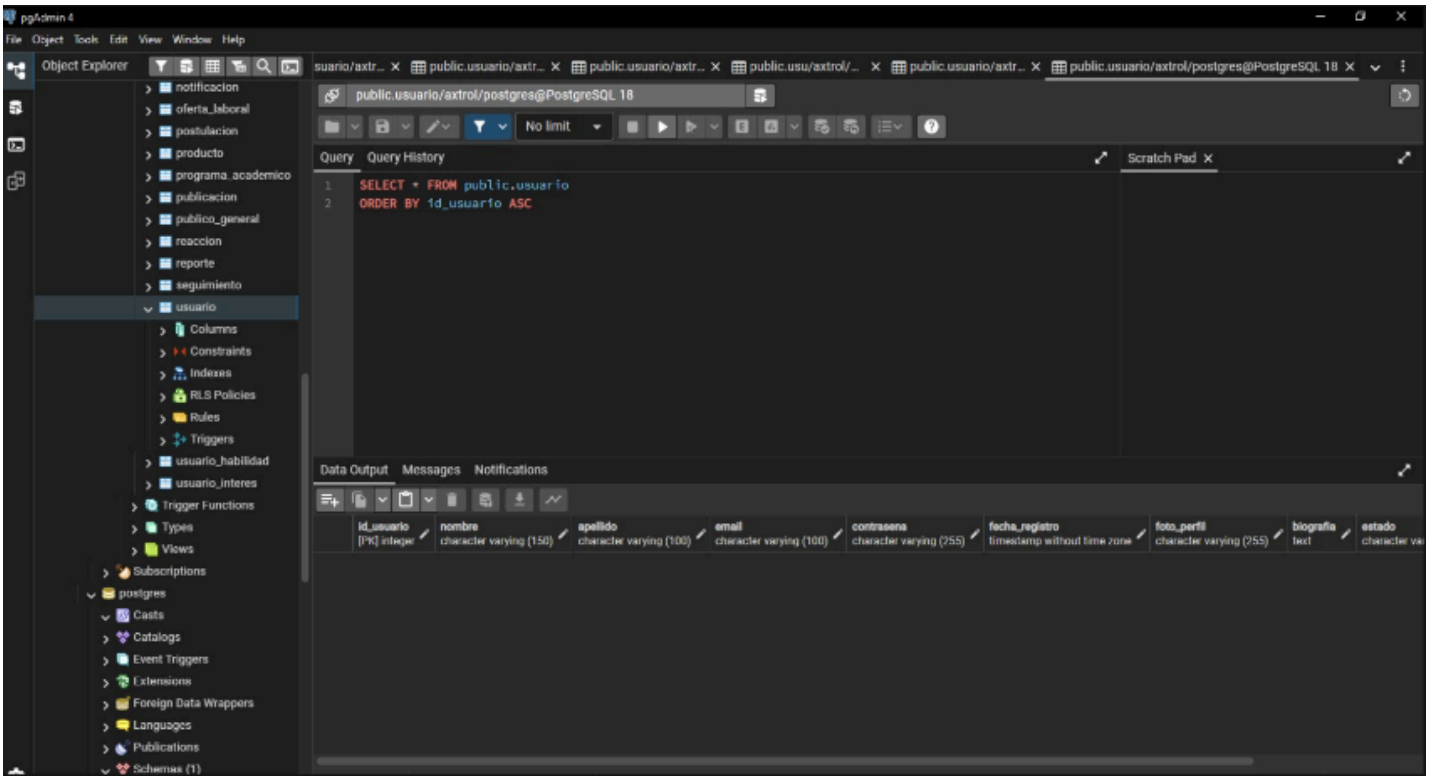
Asimismo, se utilizó el tipo JSONB en campos que requieren almacenar datos flexibles, como contenido multimedia y requisitos, permitiendo una mayor adaptabilidad en la estructura.

Finalmente, se implementaron acciones referenciales como ON DELETE CASCADE en algunas relaciones, con el fin de mantener la integridad de los datos al eliminar registros relacionados automáticamente.

DICCIONARIO DE DATOS FÍSICO			
Inventario de Tablas			
#	Tabla	Descripción	Observaciones
1	usuario	Entidad principal	Supertipo
2	estudiante	Especialización	ISA
3	docente	Especialización	ISA
4	empleado	Especialización	ISA
5	egresado	Especialización	ISA
6	publico_general	Especialización	ISA
7	programa_academico	Programas	1:N
8	publicacion	Publicaciones	Social
9	comentario	Comentarios	Recursivo
10	reaccion	Reacciones	Restricción
11	grupo	Grupos	Comunidad
12	miembro_grupo	Relación	N:M
13	evento	Eventos	Comunidad
14	asistencia_evento	Relación	N:M
15	producto	Marketplace	Comercio
16	intercambio	Transacciones	Relación
17	empresa	Empresas	Independiente
18	oferta_laboral	Vacantes	Empleo
19	postulacion	Relación	N:M
20	mensaje	Chat	Comunicación
21	notificacion	Alertas	Sistema
22	reporte	Moderación	Seguridad

5.- Lenguaje de Definición de Dato (DDL) - Scripts de CREACIÓN de Bases de Datos

Para la implementación del modelo físico se desarrolló un script de creación en lenguaje SQL que permite generar todas las tablas de la base de datos. El script fue organizado iniciando con tablas independientes como usuario, programa_academico, habilidad e interés, seguidas de tablas dependientes con claves foráneas. Se definieron claves primarias mediante SERIAL y relaciones mediante claves foráneas, asegurando la integridad de los datos. El script fue ejecutado en PostgreSQL utilizando pgAdmin4, verificando su correcto funcionamiento sin errores.



CREACIÓN DE TABLAS

USUARIOS

```
CREATE TABLE usuario (
    id_usuario SERIAL PRIMARY KEY,
    nombre VARCHAR(100),
    apellido VARCHAR(100),
    email VARCHAR(100) UNIQUE,
    contrasena VARCHAR(255),
```

```
    fecha_registro TIMESTAMP,
    foto_perfil VARCHAR(255),
    biografia TEXT,
    estado VARCHAR(20)
);

CREATE TABLE programa_academico (
    id_programa SERIAL PRIMARY KEY,
    nombre VARCHAR(100),
    facultad VARCHAR(100),
    nivel VARCHAR(50)
);

-- Subtipos
CREATE TABLE estudiante (
    id_usuario INT PRIMARY KEY,
    codigo_estudiante VARCHAR(50),
    semestre INT,
    id_programa INT,
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario) ON DELETE CASCADE,
    FOREIGN KEY (id_programa) REFERENCES programa_academico(id_programa)
);

CREATE TABLE docente (
    id_usuario INT PRIMARY KEY,
    codigo_docente VARCHAR(50),
    departamento VARCHAR(100),
    especialidad VARCHAR(100),
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario) ON DELETE CASCADE
);

CREATE TABLE empleado (
    id_usuario INT PRIMARY KEY,
    codigo_empleado VARCHAR(50),
    cargo VARCHAR(100),
    dependencia VARCHAR(100),
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario) ON DELETE CASCADE
);
```

```
CREATE TABLE egresado (
   id_usuario INT PRIMARY KEY,
   anio_egreso INT,
   empresa_actual VARCHAR(100),
   cargo_actual VARCHAR(100),
   FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario) ON DELETE CASCADE
);

CREATE TABLE publico_general (
   id_usuario INT PRIMARY KEY,
   ocupacion VARCHAR(100),
   FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario) ON DELETE CASCADE
);

 HABILIDADES E INTERESES
CREATE TABLE habilidad (
   id_habilidad SERIAL PRIMARY KEY,
   nombre VARCHAR(100),
   categoria VARCHAR(50)
);

CREATE TABLE interes (
   id_interes SERIAL PRIMARY KEY,
   nombre VARCHAR(100),
   categoria VARCHAR(50)
);

CREATE TABLE usuario_habilidad (
   id_usuario INT,
   id_habilidad INT,
   PRIMARY KEY (id_usuario, id_habilidad),
   FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),
   FOREIGN KEY (id_habilidad) REFERENCES habilidad(id_habilidad)
);

CREATE TABLE usuario_interes (
   id_usuario INT,
   id_interes INT,
   PRIMARY KEY (id_usuario, id_interes),
```

```
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),
    FOREIGN KEY (id_interes) REFERENCES interes(id_interes)
);
```

SOCIAL

```
CREATE TABLE grupo (
    id_grupo SERIAL PRIMARY KEY,
    nombre VARCHAR(100),
    descripcion TEXT,
    fecha_creacion TIMESTAMP,
    tipo VARCHAR(50),
    visibilidad VARCHAR(20),
    id_creador INT,
    FOREIGN KEY (id_creador) REFERENCES usuario(id_usuario)
);

CREATE TABLE publicacion (
    id_publicacion SERIAL PRIMARY KEY,
    contenido_texto TEXT,
    contenido_multimedia JSONB,
    fecha_publicacion TIMESTAMP,
    tipo VARCHAR(50),
    visibilidad VARCHAR(20),
    id_usuario INT,
    id_grupo INT,
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),
    FOREIGN KEY (id_grupo) REFERENCES grupo(id_grupo)
);

CREATE TABLE comentario (
    id_comentario SERIAL PRIMARY KEY,
    contenido TEXT,
    fecha_comentario TIMESTAMP,
    id_usuario INT,
    id_publicacion INT,
    id_padre INT,
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),
    FOREIGN KEY (id_publicacion) REFERENCES publicacion(id_publicacion),
    FOREIGN KEY (id_padre) REFERENCES comentario(id_comentario)
```

```
);

CREATE TABLE reaccion (
   id_reaccion SERIAL PRIMARY KEY,
   tipo_reaccion VARCHAR(20),
   fecha TIMESTAMP,
   id_usuario INT,
   id_publicacion INT,
   FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),
   FOREIGN KEY (id_publicacion) REFERENCES publicacion(id_publicacion),
   UNIQUE (id_usuario, id_publicacion)
);

CREATE TABLE seguimiento (
   id_seguidor INT,
   id_seguido INT,
   fecha_seguimiento TIMESTAMP,
   PRIMARY KEY (id_seguidor, id_seguido),
   FOREIGN KEY (id_seguidor) REFERENCES usuario(id_usuario),
   FOREIGN KEY (id_seguido) REFERENCES usuario(id_usuario)
);

COMUNIDADES
CREATE TABLE miembro_grupo (
   id_usuario INT,
   id_grupo INT,
   rol VARCHAR(20),
   fecha_ingreso TIMESTAMP,
   PRIMARY KEY (id_usuario, id_grupo),
   FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),
   FOREIGN KEY (id_grupo) REFERENCES grupo(id_grupo)
);

CREATE TABLE evento (
   id_evento SERIAL PRIMARY KEY,
   nombre VARCHAR(100),
   descripcion TEXT,
   fecha_inicio TIMESTAMP,
   fecha_fin TIMESTAMP,
```

```
    ubicacion VARCHAR(255),
    tipo VARCHAR(50),
    capacidad_maxima INT,
    id_grupo INT,
    FOREIGN KEY (id_grupo) REFERENCES grupo(id_grupo)
);

CREATE TABLE asistencia_evento (
    id_usuario INT,
    id_evento INT,
    estado VARCHAR(20),
    fecha_registro TIMESTAMP,
    PRIMARY KEY (id_usuario, id_evento),
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),
    FOREIGN KEY (id_evento) REFERENCES evento(id_evento)
);

 MARKETPLACE
CREATE TABLE producto (
    id_producto SERIAL PRIMARY KEY,
    nombre VARCHAR(100),
    descripcion TEXT,
    categoria VARCHAR(50),
    estado VARCHAR(20),
    tipo_intercambio VARCHAR(20),
    precio DECIMAL(10,2),
    fecha_publicacion TIMESTAMP,
    fotos JSONB,
    id_vendedor INT,
    FOREIGN KEY (id_vendedor) REFERENCES usuario(id_usuario)
);

CREATE TABLE intercambio (
    id_intercambio SERIAL PRIMARY KEY,
    id_producto INT,
    id_comprador INT,
    fecha TIMESTAMP,
    estado VARCHAR(20),
    FOREIGN KEY (id_producto) REFERENCES producto(id_producto),
```

```
    FOREIGN KEY (id_comprador) REFERENCES usuario(id_usuario)
);
```

EMPLEO

```
CREATE TABLE empresa (
    id_empresa SERIAL PRIMARY KEY,
    nombre VARCHAR(100),
    sector VARCHAR(100),
    descripcion TEXT,
    sitio_web VARCHAR(255),
    contacto VARCHAR(100)
);

CREATE TABLE oferta_laboral (
    id_oferta SERIAL PRIMARY KEY,
    titulo VARCHAR(100),
    descripcion TEXT,
    requisitos JSONB,
    salario DECIMAL(10,2),
    modalidad VARCHAR(50),
    fecha_publicacion TIMESTAMP,
    fecha_cierre TIMESTAMP,
    id_empresa INT,
    FOREIGN KEY (id_empresa) REFERENCES empresa(id_empresa)
);

CREATE TABLE postulacion (
    id_usuario INT,
    id_oferta INT,
    fecha_postulacion TIMESTAMP,
    estado VARCHAR(20),
    cv_adjunto VARCHAR(255),
    PRIMARY KEY (id_usuario, id_oferta),
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),
    FOREIGN KEY (id_oferta) REFERENCES oferta_laboral(id_oferta)
);
```

COMUNICACIÓN

```
CREATE TABLE mensaje (
```

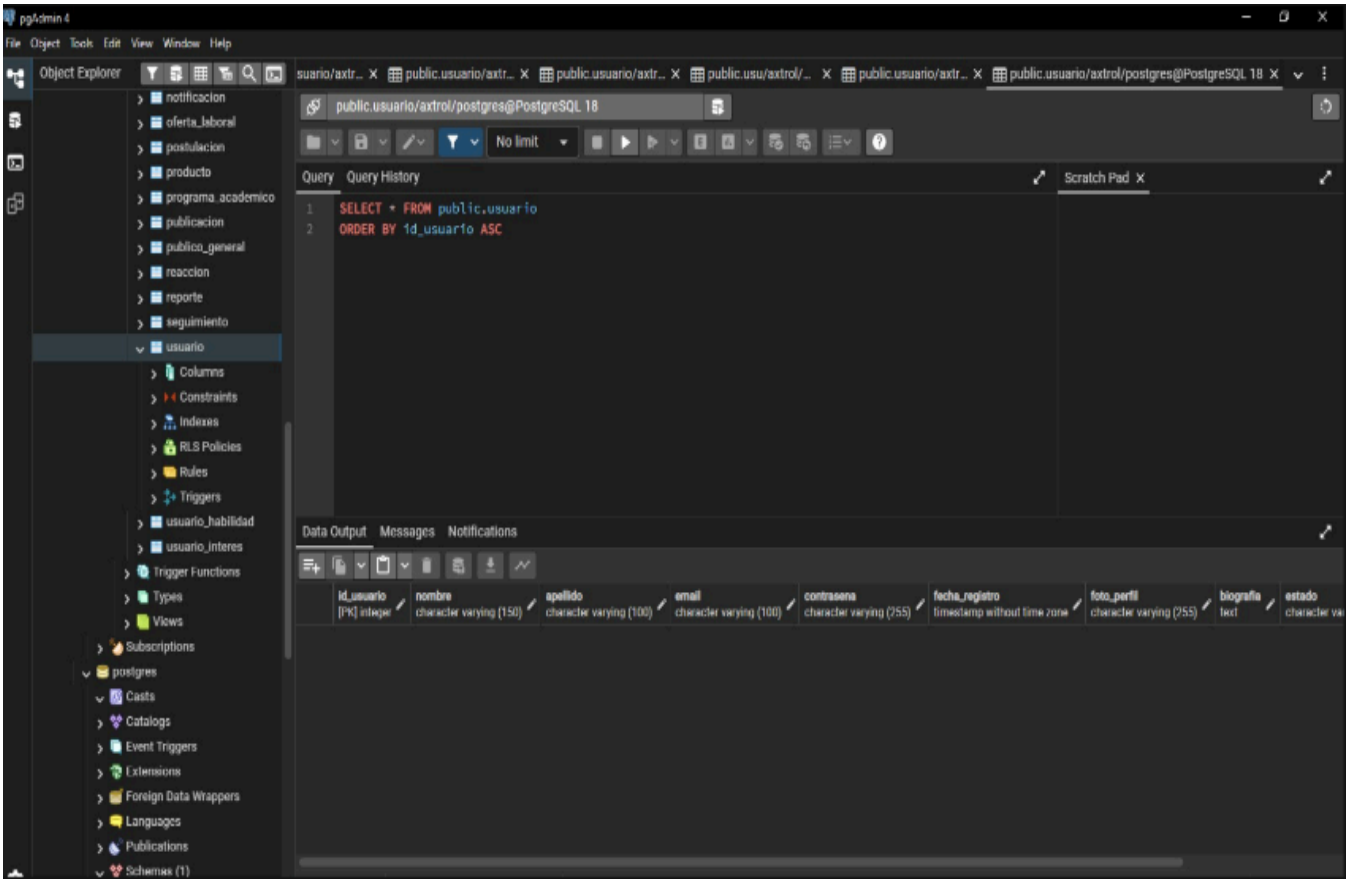
```
    id_mensaje SERIAL PRIMARY KEY,
    contenido TEXT,
    fecha_envio TIMESTAMP,
    leido BOOLEAN,
    id_emisor INT,
    id_receptor INT,
    FOREIGN KEY (id_emisor) REFERENCES usuario(id_usuario),
    FOREIGN KEY (id_receptor) REFERENCES usuario(id_usuario)
);

CREATE TABLE notificacion (
    id_notificacion SERIAL PRIMARY KEY,
    tipo VARCHAR(50),
    contenido TEXT,
    fecha TIMESTAMP,
    leida BOOLEAN,
    id_usuario INT,
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
);

CREATE TABLE reporte (
    id_reporte SERIAL PRIMARY KEY,
    motivo VARCHAR(50),
    descripcion TEXT,
    fecha TIMESTAMP,
    estado VARCHAR(20),
    id_reportante INT,
    FOREIGN KEY (id_reportante) REFERENCES usuario(id_usuario)
);
```

6.- Lenguaje de Definición de Dato (DDL) - Scripts de MODIFICACIÓN de Bases de Datos

Se desarrolló un script de modificación utilizando sentencias ALTER TABLE para realizar cambios en la estructura de la base de datos. En este proceso se agregaron nuevos campos, se modificaron tipos de datos, se renombraron tablas y se aplicaron restricciones como CHECK y UNIQUE.

```
-- MODIFICACIONES (DDL)


ALTER TABLE usuario ADD COLUMN telefono VARCHAR(20);
ALTER TABLE usuario ALTER COLUMN nombre TYPE VARCHAR(150);

CREATE TABLE curso (
   id_curso SERIAL PRIMARY KEY
);

ALTER TABLE curso
ADD COLUMN nombre VARCHAR(100),
ADD COLUMN creditos INT,
ADD COLUMN descripcion TEXT;

ALTER TABLE curso DROP COLUMN descripcion;
ALTER TABLE curso RENAME TO asignatura;

ALTER TABLE asignatura
ADD COLUMN codigo VARCHAR(20) UNIQUE;

ALTER TABLE asignatura
ADD COLUMN fecha_inicio DATE,
ADD COLUMN fecha_fin DATE,
ADD CONSTRAINT chk_fechas CHECK (fecha_fin >= fecha_inicio);

ALTER TABLE asignatura
ADD COLUMN cupos INT,
ADD CONSTRAINT chk_cupos CHECK (cupos >= 0);

ALTER TABLE asignatura
ALTER COLUMN nombre TYPE VARCHAR(200);
```

```
ALTER TABLE asignatura
ADD CONSTRAINT chk_creditos CHECK (creditos BETWEEN 1 AND 10);

CREATE INDEX idx_nombre_asignatura ON asignatura(nombre);

ALTER TABLE asignatura DROP COLUMN fecha_fin;

TRUNCATE TABLE asignatura;
```



7.- Análisis de resultados

El desarrollo de esta actividad nos permitió comprender de manera más clara la importancia de estructurar adecuadamente una base de datos desde sus primeras etapas de diseño. A lo largo del proceso, se evidenció que no solo es necesario identificar correctamente las entidades y atributos, sino también garantizar la coherencia entre el modelo conceptual, lógico y físico.

Uno de los aspectos más relevantes fue la selección adecuada de los tipos de datos dentro del SGBD. Elegir entre tipos como VARCHAR, TEXT o INT no es algo arbitrario, ya que influye directamente en el almacenamiento, la eficiencia y el rendimiento del sistema. Por ejemplo, el uso de VARCHAR permite establecer un límite en la longitud de los datos, lo cual hace que se optimice mejor el espacio y evita el almacenamiento innecesario de información.

Asimismo, la implementación de claves primarias y foráneas resultó fundamental para asegurar la integridad de los datos. Las claves primarias permiten identificar de forma única cada registro dentro de una tabla, mientras que las claves foráneas establecen relaciones entre las tablas, permitiendo representar de manera adecuada la interacción entre los diferentes componentes del sistema.

Otro aspecto importante fue la aplicación de restricciones como NOT NULL, UNIQUE y CHECK, las cuales ayudan a mantener la calidad de la información almacenada y a prevenir inconsistencias en el trabajo. Además, el uso de acciones referenciales como ON DELETE CASCADE permite mantener la coherencia de la base de datos al eliminar registros relacionados automáticamente.

En general, esta actividad permitió fortalecer el conocimiento en el uso del lenguaje SQL y en la implementación de bases de datos en PostgreSQL, nos ayudo a reforzar los conocimientos y a hacernos entender mejor como equipo ayudándonos a evidenciar la importancia de un diseño estructurado y bien definido para el desarrollo de sistemas en la vida profesional.

8.- Reflexiones y conclusiones individuales

Yuli Tatiana Blanco Nieto:

Durante el desarrollo de esta actividad logré comprender mejor el proceso de construcción de una base de datos, desde la parte teórica hasta su implementación práctica. Al inicio, algunos conceptos relacionados con el modelo lógico y físico me generaban dudas, especialmente en lo que tiene que ver con las relaciones entre tablas y la forma en que se representan en el sistema. Sin embargo, a medida que avanzaba en el trabajo, fui entendiendo mejor cómo se conectan todos estos elementos.

Este ejercicio me permitió reforzar mis conocimientos en SQL y entender de manera más clara cómo se organiza la información dentro de una base de datos. También comprendí que no se trata solo de crear tablas, sino de diseñar una estructura que sea coherente, eficiente y que permita manejar correctamente los datos en un sistema real.

En cuanto al trabajo en equipo, fue una experiencia interesante, ya que implicó coordinación y comunicación para dividir las tareas y avanzar de manera organizada. Aunque en algunos momentos hubo dificultades para sincronizar el trabajo, se logró avanzar y cumplir con los objetivos planteados.

Además, considero que esta actividad fue muy útil para mi formación académica, ya que me permitió aplicar los conocimientos vistos en clase en un contexto más práctico. Esto ayuda no solo a entender mejor los temas, sino también a prepararse para situaciones reales en el ámbito profesional.

En conclusión, esta experiencia me permitió fortalecer mis habilidades en bases de datos y comprender la importancia que tienen dentro del desarrollo de software.

Juan David Restrepo Bedoya:

En este trabajo pude comprender mejor cómo se construye una base de datos desde cero utilizando el modelo entidad-relación. Al inicio se me hizo complicado diferenciar entre entidades, atributos y relaciones, ya que varias cosas parecían encajar en más de una categoría. Sin embargo, a medida que avanzaba el proyecto, fui entendiendo cómo organizar la información de forma más clara y lógica.

Uno de los aprendizajes más importantes fue definir correctamente las cardinalidades y reconocer cuándo era necesario usar tablas intermedias para resolver relaciones muchos a muchos, como en los casos de grupos, habilidades o postulaciones. Esto me ayudó a ver la importancia de evitar redundancias y mantener la integridad de los datos.

También fue clave trabajar con la entidad usuario y sus subtipos, ya que permitió organizar distintos tipos de personas dentro del sistema sin repetir información innecesaria. Esto hizo que el modelo fuera más ordenado y fácil de entender.

La parte de llevar el modelo a PostgreSQL fue muy útil, porque permitió aplicar todo lo aprendido de forma práctica. Crear las tablas, definir claves primarias y foráneas, y ejecutar los scripts me ayudó a ver cómo el diseño se convierte en algo funcional dentro de un sistema real.

En general, este proyecto me permitió mejorar mi forma de analizar problemas y estructurar soluciones relacionadas con bases de datos. Aunque al principio hubo errores y confusiones, el proceso ayudó a entender mejor cómo funcionan los sistemas y la importancia de hacer un buen diseño desde el inicio para evitar problemas en el futuro.

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas Digitales
Base de Datos I (SD1006)

Rúbrica: Criterios de Evaluación de la Tarea

#	Ítems Tarea		Peso	Cal
1	Corrección Diagrama Entidad-Relación de Chen		10	
2	Inventario de tablas resultantes del Modelo Lógico		10	
3	Tabla Comparativa de tipos de Datos		10	
4	Diccionario de Datos Físico (PostgreSQL)		80	
5	DDL - Scripts de CREACIÓN de Bases de Datos		120	
6	DDL - Scripts de MODIFICACIÓN de Bases de Datos		30	
7	Análisis de resultados (en equipo - 300 palabras mínimo)		20	
8	Reflexiones y Conclusiones individuales (300 palabras mínimo)		30	
9	Informe: Estructura, Contenido y Calidad de presentación		30	
10	Video de Sustentación		100	
11	Repositorio GIT		20	
12	Participación en Foro de Tarea		40	
	NOTA = xx/500 =	Total	500	